

PythonStyle

中文 saturo

June 2026

目录

1	实验一	1
1.1	1. 基于 tkinter 的学生成绩录入与统计系统的设计与实现 . . .	1
1.1.1	实验目的	1
1.1.2	实验内容	1
1.2	实验要求	1
1.2.1	详细设计	2
1.2.2	基于 opencv 车牌识别系统	2
2	实验四	18
2.1	1. 基于 opencv 车牌识别系统	18
2.1.1	实验目的	18
2.1.2	实验内容及要求	19
2.1.3	详细设计	19
2.1.4	基于 opencv 车牌识别系统	19

1 实验一

1.1 1. 基于 tkinter 的学生成绩录入与统计系统的设计与实现

1.1.1 实验目的

1. 熟悉 Python 标准库 tkinter 创建 GUI 应用程序的方法和步骤。
2. 熟悉 Label、Entry、Button、Treeview、MessageBox 等常用 tkinter 组件的使用方法。
3. 熟悉 CSV 文件的读取与写入方法，掌握简单数据持久化操作。
4. 掌握列表、字典、函数封装、异常处理等 Python 基础知识在实际问题中的应用。

1.1.2 实验内容

设计并实现一个基于 tkinter 的学生成绩录入与统计系统。程序应提供图形化操作界面，用户可以在界面中录入学生成绩信息，并对已录入的数据进行查看、删除、保存和统计。系统中每条学生成绩记录至少包含学号、姓名和成绩三个字段。用户可以通过输入框录入学生信息，点击按钮后将数据添加到表格中显示。系统应能够将当前成绩数据保存到本地 CSV 文件中，并在程序再次运行时读取已有数据。系统还应提供简单统计功能，例如计算平均分、最高分、最低分、及格人数和不及格人数。

1.2 实验要求

1. 使用 tkinter 设计图形化界面，界面布局清晰，操作按钮完整。
2. 每条成绩记录至少包含学号、姓名和成绩三个字段。
3. 实现成绩信息的添加、删除、显示、保存和读取功能。
4. 使用 CSV 文件保存成绩数据，程序关闭后再次运行时能够读取已有数据。
5. 实现成绩统计功能，包括平均分、最高分、最低分、及格人数和不及格人数。

6. 进行必要的输入合法性检查，例如学号和姓名不能为空，成绩必须为 0 到 100 之间的数字。
7. 代码结构清晰，主要功能应封装为函数或类，并添加必要注释。
8. 运行结果中应展示正常输入、错误输入、数据保存、数据读取和统计结果等测试情况。

1.2.1 详细设计

源代码（带注释）

1.2.2 基于 opencv 车牌识别系统

代码：

```
"""
基于 tkinter 的学生成绩录入与统计系统
Author: wu-asia
"""

import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
import csv
import os

class StudentScoreSystem:

    def __init__(self):

        # 创建主窗口

        self.root = tk.Tk()
        self.root.title("学生成绩录入与统计系统")
        self.root.geometry("800x600")
```

```

self.root.resizable(False, False)
# 数据列表
self.students = []

# 创建界面
self.create_widgets()

# 读取CSV数据（后续实现）
self.load_data()

# 显示数据
self.refresh_tree()

# 创建界面

def create_widgets(self):
    # 输入区域
    input_frame = tk.LabelFrame(
        self.root,
        text="学生信息录入",
        padx=10,
        pady=10
    )

    input_frame.pack(fill="x", padx=10, pady=10)

    # 学号
    tk.Label(
        input_frame,
        text="学号: "
    ).grid(row=0, column=0, padx=5, pady=5)

    self.entry_id = tk.Entry(

```

```

        input_frame,
        width=20
    )
    self.entry_id.grid(row=0, column=1, padx=5)

    # 姓名
    tk.Label(
        input_frame,
        text="姓名: "
    ).grid(row=0, column=2, padx=5)

    self.entry_name = tk.Entry(
        input_frame,
        width=20
    )
    self.entry_name.grid(row=0, column=3, padx=5)

    # 成绩
    tk.Label(
        input_frame,
        text="成绩: "
    ).grid(row=0, column=4, padx=5)

    self.entry_score = tk.Entry(
        input_frame,
        width=20
    )

    self.entry_score.grid(row=0, column=5, padx=5)

    # 按钮区域
    button_frame = tk.Frame(self.root)

```

```

button_frame.pack(pady=10)

tk.Button(
    button_frame,
    text="添加",
    width=12,
    command=self.add_student
).grid(row=0, column=0, padx=10)

tk.Button(
    button_frame,
    text="删除",
    width=12,
    command=self.delete_student
).grid(row=0, column=1, padx=10)

tk.Button(
    button_frame,
    text="保存",
    width=12,
    command=self.save_data
).grid(row=0, column=2, padx=10)

tk.Button(
    button_frame,
    text="统计",
    width=12,
    command=self.statistics
).grid(row=0, column=3, padx=10)

# Treeview区域

```

```

tree_frame = tk.Frame(self.root)

tree_frame.pack(fill="both", expand=True, padx=10)

columns = (
    "id",
    "name",
    "score"
)

self.tree = ttk.Treeview(
    tree_frame,
    columns=columns,
    show="headings",
    height=15
)

self.tree.heading(
    "id",
    text="学号"
)

self.tree.heading(
    "name",
    text="姓名"
)

self.tree.heading(
    "score",
    text="成绩"
)

self.tree.column(

```



```

        "id",
        width=180,
        anchor="center"
    )

    self.tree.column(
        "name",
        width=180,
        anchor="center"
    )

    self.tree.column(
        "score",
        width=120,
        anchor="center"
    )

    # 滚动条
    scrollbar = ttk.Scrollbar(
        tree_frame,
        orient="vertical",
        command=self.tree.yview
    )

    self.tree.configure(
        yscrollcommand=scrollbar.set
    )

    self.tree.pack(
        side="left",
        fill="both",
        expand=True
    )

```

```

scrollbar.pack(
    side="right",
    fill="y"
)

# 统计信息区域

stat_frame = tk.LabelFrame(
    self.root,
    text="统计信息"
)

stat_frame.pack(
    fill="x",
    padx=10,
    pady=10
)

self.label_avg = tk.Label(
    stat_frame,
    text="平均分: 0"
)

self.label_avg.grid(
    row=0,
    column=0,
    padx=20,
    pady=5
)

self.label_max = tk.Label(
    stat_frame,

```

```

        text="最高分: 0"
    )

    self.label_max.grid(
        row=0,
        column=1,
        padx=20
    )

    self.label_min = tk.Label(
        stat_frame,
        text="最低分: 0"
    )

    self.label_min.grid(
        row=0,
        column=2,
        padx=20
    )

    self.label_pass = tk.Label(
        stat_frame,
        text="及格人数: 0"
    )

    self.label_pass.grid(
        row=1,
        column=0,
        padx=20,
        pady=5
    )

    self.label_fail = tk.Label(

```

```

        stat_frame,
        text="不及格人数: 0"
    )

    self.label_fail.grid(
        row=1,
        column=1,
        padx=20
    )

# 以下函数将在第二部分实现

# 增加学生
def add_student(self):

    # 获取输入内容
    stu_id = self.entry_id.get().strip()
    name = self.entry_name.get().strip()
    score = self.entry_score.get().strip()

    #输入合法性检查
    if stu_id == "":
        messagebox.showerror("输入错误", "学号不能为空!")
        return

    if name == "":
        messagebox.showerror("输入错误", "姓名不能为空!")
        return

    if score == "":
        messagebox.showerror("输入错误", "成绩不能为空!")
        return

```

```

# 成绩必须是数字
try:
    score = float(score)
except ValueError:
    messagebox.showerror("输入错误", "成绩必须是数字！")
    return

# 成绩范围检查
if score < 0 or score > 100:
    messagebox.showerror("输入错误", "成绩必须在0~100之间！")
    return

# 学号不能重复
for stu in self.students:
    if stu["id"] == stu_id:
        messagebox.showerror("输入错误", "该学号已经存在！")
        return

# 添加数据
self.students.append(
    {
        "id": stu_id,
        "name": name,
        "score": score
    }
)

# 刷新表格
self.refresh_tree()

# 清空输入框
self.entry_id.delete(0, tk.END)

```

```

self.entry_name.delete(0, tk.END)
self.entry_score.delete(0, tk.END)

messagebox.showinfo("成功", "学生成绩添加成功！")

#删除学生
def delete_student(self):

    # 获取当前选中的记录
    selected = self.tree.selection()

    if not selected:
        messagebox.showwarning("提示", "请先选择需要删除的记录！")
        return

    # 是否确认删除
    result = messagebox.askyesno(
        "确认删除",
        "确定删除该条成绩记录吗？"
    )

    if not result:
        return

    # 获取学号
    item = self.tree.item(selected[0])

    values = item["values"]

    stu_id = str(values[0])

    # 删除列表中的数据
    for stu in self.students:

```

```

        if stu["id"] == stu_id:
            self.students.remove(stu)
            break

    # 刷新表格
    self.refresh_tree()

    messagebox.showinfo("成功", "删除成功！")

#刷新TreeView
def refresh_tree(self):

    # 删除Treeview中所有数据
    for item in self.tree.get_children():
        self.tree.delete(item)

    # 重新插入全部数据
    for stu in self.students:

        self.tree.insert(
            "",
            tk.END,
            values=(
                stu["id"],
                stu["name"],
                stu["score"]
            )
        )

def save_data(self):

    try:

```

```

with open(
    "students.csv",
    "w",
    newline="",
    encoding="utf-8-sig"
) as file:

    writer = csv.writer(file)

    # 写入表头
    writer.writerow(
        ["学号", "姓名", "成绩"]
    )

    # 写入数据
    for stu in self.students:

        writer.writerow(
            [
                stu["id"],
                stu["name"],
                stu["score"]
            ]
        )

    messagebox.showinfo(
        "保存成功",
        "成绩数据已保存到 students.csv"
    )

except Exception as e:

    messagebox.showerror(

```



```

        "保存失败",
        str(e)
    )

def load_data(self):

    # 文件不存在直接返回
    if not os.path.exists("students.csv"):
        return

    try:

        with open(
            "students.csv",
            "r",
            newline="",
            encoding="utf-8-sig"
        ) as file:

            reader = csv.reader(file)

            # 跳过标题行
            next(reader, None)

            self.students.clear()

            for row in reader:

                if len(row) != 3:
                    continue

                self.students.append(
                    {

```

```

        "id": row[0],
        "name": row[1],
        "score": float(row[2])
    }
)

except Exception as e:

    messagebox.showerror(
        "读取失败",
        str(e)
    )

def statistics(self):

    if len(self.students) == 0:

        messagebox.showwarning(
            "提示",
            "暂无学生成绩数据！"
        )

        return

    scores = []

    for stu in self.students:

        scores.append(
            stu["score"]
        )

    average = sum(scores) / len(scores)

```

```
maximum = max(scores)

minimum = min(scores)

pass_num = 0

fail_num = 0

for score in scores:

    if score >= 60:
        pass_num += 1
    else:
        fail_num += 1

self.label_avg.config(
    text=f"平均分: {average:.2f}"
)

self.label_max.config(
    text=f"最高分: {maximum:.2f}"
)

self.label_min.config(
    text=f"最低分: {minimum:.2f}"
)

self.label_pass.config(
    text=f"及格人数: {pass_num}"
)

self.label_fail.config(
```

```

        text=f"不及格人数: {fail_num}"
    )

    messagebox.showinfo(
        "统计完成",
        "成绩统计已更新！"
    )

# 启动程序
def run(self):
    self.root.mainloop()

# 主函数
if __name__ == "__main__":

    app = StudentScoreSystem()

    app.run()

```

2 实验四

2.1 1. 基于 opencv 车牌识别系统

2.1.1 实验目的

针对静态车牌图像完成完整车牌识别流程开发。借助 OpenCV 与 numpy 完成图像灰度化、滤波去噪、边缘检测、形态学运算、轮廓筛选等图像预处理操作，定位图片内车牌区域并完成字符分割；对分割后的字符进行特征匹配识别，输出车牌完整号码；利用 tkinter 搭建简易可视化交互界面，实现图片导入、图像处理、识别结果实时展示功能，搭建一套轻量化本地车牌图像识别工具，掌握数字图像处理与目标字符识别基础流程。实验数据：各类含车牌静态图片，包含不同光照、拍摄角度的车辆车牌图像作为识别测试样本：<https://tianchi.aliyun.com/dataset/203402>。

2.1.2 实验内容及要求

1. 开发语言与核心库限定：Python、numpy、opencv-python;
2. 完成图像预处理、车牌定位、字符分割、字符识别整套图像处理算法编写;
3. 使用 tkinter 设计图形交互界面，支持本地车牌图片上传、原图显示、处理中间效果图、最终识别结果展示;
4. 程序模块化设计，图像预处理、车牌定位、字符识别、界面功能分开封装，代码注释清晰可运行;
5. 可加载本地车牌图片完成识别，界面直观输出识别到的车牌字符。

2.1.3 详细设计

源代码（带注释）

2.1.4 基于 opencv 车牌识别系统

代码：